DOMAIN 3 · BUILD AGENTS

A topic is one conversation behaviour.

A **topic** is the unit a Copilot Studio agent uses to handle one recognisable user intent — its trigger, its branches, its questions, its actions, its exits, all bundled together and named.

INSIDE ONE TOPIC, YOU WILL FIND

01
Trigger phrases
5–10 phrases that decide whether the topic starts.

EXAMPLES
"refund status"
"how do returns work"
"I want my money back"

02
Questions & slot filling
Each Question carries an entity that fills when the user speaks it.

DEFINES
Order #? · Email? · Date? · City?

03
Conditions & branches
Switch on a variable's value with Power Fx.

EVALUATES
`If Status == "VIP" · If Total > $1000`

04
Actions & exits
Call a connector, write to a table, escalate, end.

ENDS WITH
A response, a redirect, or an escalate.

A topic is a tree of nodes — a trigger at the root, questions and branches in the middle, an exit at the leaf.

ONE TOPIC, ONE PROMISE

Five topics, each with one job, beats one topic trying to do five.

When a topic grows past twenty nodes, the right move is rarely more conditions — it's a second topic. Split on user intent, not on edge case.

- **Topics are addressed by name in tests.**
- **Topics can call other topics; an agent has many.**
- **A topic without a name is hard to debug.**

TOPICS · TRIGGER PHRASES

A trigger phrase is a door, not a contract.

A **trigger phrase** is one of 5–10 example utterances that helps the orchestrator decide whether this topic is the right one. Phrases are evidence, not the only path in.

ANATOMY

A phrase a real user would say

Yes, write it —

Real wording. The way the actual user phrases it, typos and allophones included.

Don't write —

Keywords stuffed in. Synonyms dumped without order. Phrases that all mean the same thing six times.

Vary the verb

Same intent, different verbs: "I want", "can I", "how do I", "where is".

Stay specific

If a phrase opens four topics, it's a routing problem, not a phrase problem.

A SMALL LIST

```
"refund status"
"where is my order"
"cancel a booking"
```

A LONG LIST

A list of a hundred phrases mostly duplicates itself. Quality beats count.

PER TOPIC, AIM FOR

5 – 10

phrases that real users would type.

Microsoft's worked example: a "store hours" topic with three nearby phrases + two phrasings works; fifty does not improve much.

If two topics both fire on the same utterance, prefer renaming + narrowing over adding conditions downstream — the trigger is the front door.

ORCHESTRATION · TWO MODES

Generative orchestration vs classic routing.

Two different ways to glue topics, knowledge, and actions together. Pick by outcome — not by cleverness. Both can ship production work, in different shapes.

MODE 01

Classic routing



A script — every path is a topic you wrote.

Best when: deterministic — same answer every time.

Limit: anything not in your triggers does not happen.

MODE 02

Generative orchestration



A planner — one orchestrator picks on the fly.

Best when: open-ended — the user phrases it differently each time.

Limit: opaque — the path it took is logged, not narrated.

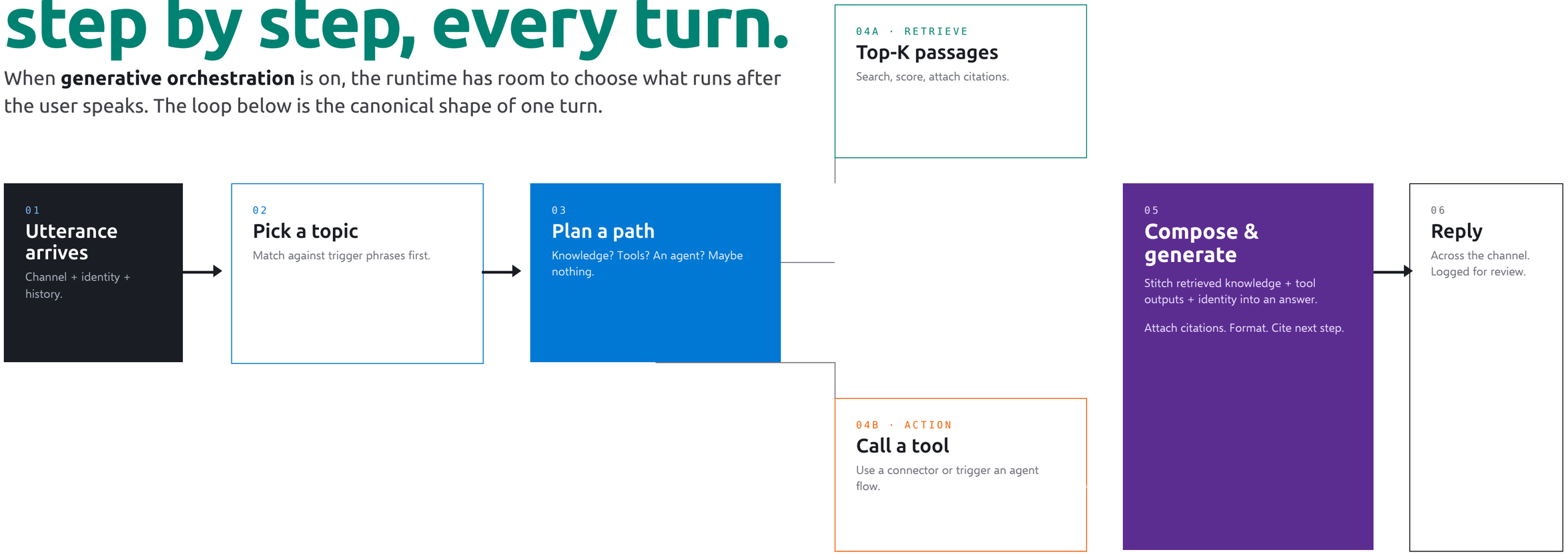
MIX THEM — DELIBERATELY

Use classic for the spine. Use generative for the open edges.

INSIDE GENERATIVE ORCHESTRATION

What the orchestrator actually does — step by step, every turn.

When **generative orchestration** is on, the runtime has room to choose what runs after the user speaks. The loop below is the canonical shape of one turn.



WHY THE LOOP CLOSES WELL

Plan + retrieve + act + generate, log every step. Generative orchestration is observable because the runtime records each decision — not because it narrates one.

POWER FX · USE IT WHERE RULES LIVE

Power Fx is the spine— it joins the parts together.

Where a behaviour is **defined or formulaic**, write a Power Fx expression. Where a behaviour is **judgement**, leave it to the orchestrator. The two don't compete — they fit.

01 · CONDITIONS

Compare values

Branch on whether a variable equals, contains, or matches a pattern.

02 · VARIABLES

Set, read, transform

Set a variable's value, parse strings, build objects, trim text.

03 · TABLES

Bind to rows

Filter rows, look up columns, decide what to write back where.

A FORMULA LOOKS LIKE THIS

COMPARING VALUES

```
If(Global.IsVIP = true,
  "escalate",
  Topic.EscalationPath)
```

BUILDING A STRING

```
"Hi " & User.FirstName &
" — order " & Topic.OrderId
& " is now " & Status.Text
```

FILTERING ROWS

```
Filter(
  Table.Rows,
  Status = "Open"
)
```

VARIABLES · THE NAMES THINGS GO BY

A variable is a name for a value the topic holds.

A **variable** is a typed slot. Pick the right scope, the right type, the right name. Most topic bugs are variable bugs.

SCOPE

Topic (local)

Lives only inside the topic

Best for: user inputs, branch flags, ephemeral values.

`Topic.OrderId`

SCOPE

Bot (global)

Shared across topics, one agent

Best for: identity, prefs, escalation flag.

`Global.IsVIP`

SCOPE

System · given by the runtime, do not rename

`System.Conversation.Id` · `System.User.Language`

THREE RULES

- Pick the narrowest scope that works.
- Use System variables, do not redefine them.
- Prefix matters — Global, Topic, System.

WATCH FOR

A Global set in one topic is read by another. A Topic set is not. A variable's name is global — pick carefully.

TYPES

String · Boolean · Number · DateTime · Record · Table · Choice

Strongly typed. Match the type to the rule.

ENTITIES & SLOT FILLING

An entity is a pattern the agent looks for in every message.

An **entity** is a named type with examples. The agent scans each user message, finds instances of the pattern, and **slot-fills** the variable that asked for it.



THREE FLAVOURS OF ENTITY

PRE-BUILT

Closed-list entities

A list the agent matches against; e.g. city names.

REGEX

Pattern-based

A regular expression; e.g. order number O-####.

CUSTOM

Your own list

Closed list you author; product names, ticket IDs.

TWO NOTES WORTH KEEPING

Slot filling is proactive — keep going

Scan every message, not just the one carrying the question. The agent finds "tomorrow at 3pm" while you ask about order status.

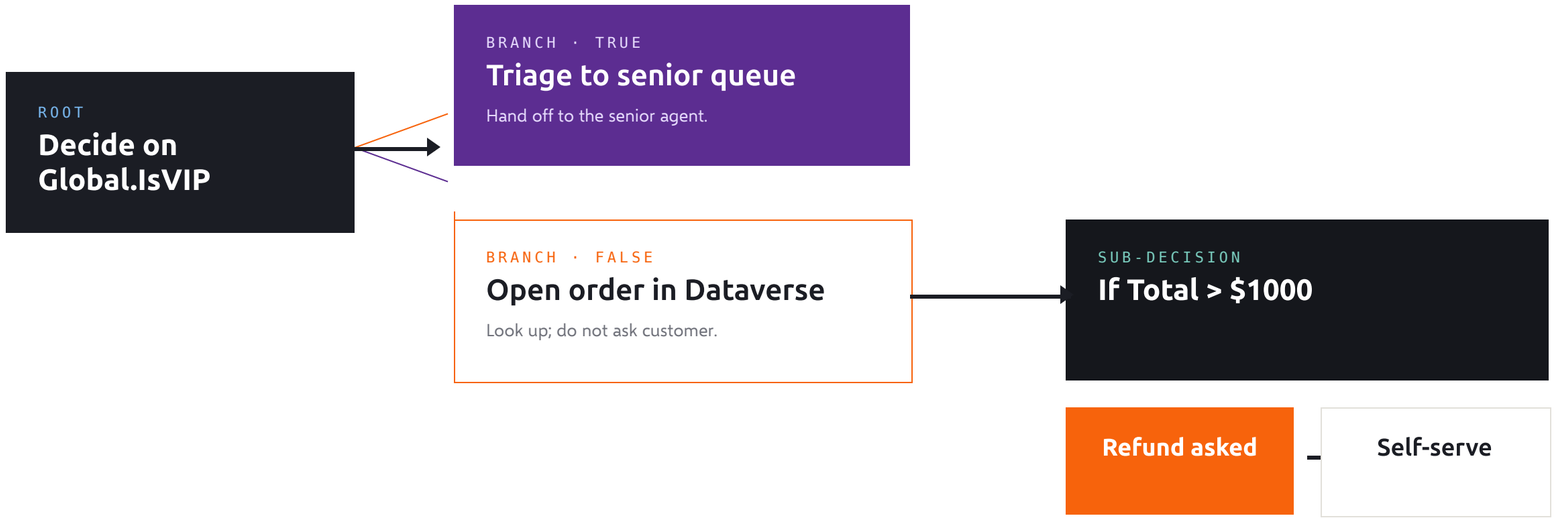
If the user does not provide it, ask

A question node takes the next-turn burden. Set the entity on the variable — if still empty, the question still fires.

CONDITIONS & BRANCHING

A branch is a decision on one variable's value.

A **condition** asks a yes/no question of a variable. Branches cluster into a small tree. Don't grow the tree beyond what one designer can hold in mind.



THREE DISCIPLINES

01 · NAME THE VARIABLE

If you can't read the test aloud, rename.

02 · MATCH ONE QUESTION

A condition answers one yes/no. Split if it doesn't.

03 · KEEP THE TREE FLAT

Five deep is too deep. Refactor at the topic boundary.

A COMMON BUG

Branches that read “Status equals any of” are slow, brittle, and the orchestrator will skip them under load. Keep the rule visible; keep the tree small.

ADAPTIVE CARDS · SNIPPETS OF UI IN A TURN

An adaptive card is a JSON UI the agent can hand back.

An **adaptive card** is a JSON payload describing a small UI surface — inputs, choices, summary — that **renders natively** in Teams, Outlook, the demo website, and a few other surfaces.

A CARD

Cancel order — please confirm

Order O-4471 will be cancelled. Refund posting takes 5–7 business days.

Confirm cancel | Keep order

Tell us a date

Input.Date (yyyy-mm-dd) · Input.Set Reminder

Summary facts

A 2-column factset — Title, Value, FactSet entries.

ANATOMY

JSON in, JSON out — the host renders.

ELEMENT
What shows up.

INPUT
Collect data.

ACTIONSET
Buttons that submit.

INPUTS BECOME VARIABLES
Action.Submit carries the input values into the next turn.

A card is for “show and ask”. A message is for “tell”. Don't put an entire conversation in a card.

SYSTEM TOPICS · THE AGENT ALWAYS CARRIES

System topics are pre-loaded — you do not author them.

A few topics arrive with the agent and cannot be deleted: **Conversation start**, **Fallback**, **Escalate**, **End of conversation**. You can author over them — but

SYSTEM · CONVERSATION START

First message of a session

Greet, invite, set the scene. Avoid blasting instructions.

SYSTEM · FALLBACK

No topic caught the utterance

The catch-all. Don't leave it empty — log, redirect, or answer.

SYSTEM · ESCALATE

Hand the conversation to a human

Generate a transfer ticket, route to a queue.

SYSTEM · END OF CONVERSATION

A clean close

End-of-session survey, summary, or quiet end.

Author over their content, but never delete the four. They are the agent's last resort.

THREE WARNINGS

Don't rename system topics — their names are how people reference them.

- Fallback runs every time nothing else fits.
- Mixing generative answers with Fallback produces duplicates.
- Localise messages — they are user-visible by default.

THE SINGLE BIGGEST RULE

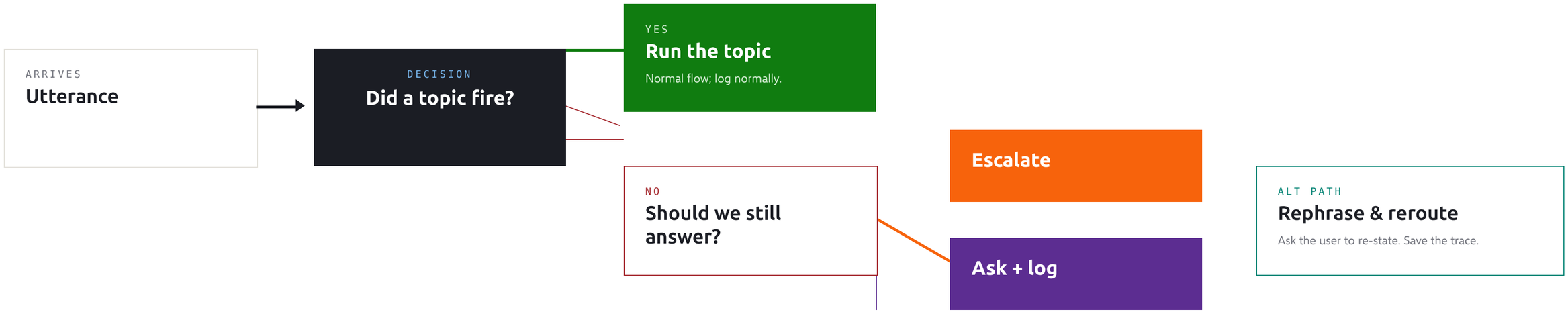
Fallback never says “I don't know”.

It says what to do next — log it, route it, or hand it to a human. Every agent needs one.

FALLBACK & ESCALATION

When nothing else fits, the fallback path is the experience.

When the orchestrator and topics miss, the user lands in a **fallback**. That fallback decides whether the user trusts the agent next time.



WHAT THE FALLBACK TOPIC OWES THE USER

An honest next step. A log. A reason that isn't “because I don't know”.

MIXING WITH GENERATIVE ANSWERS

Generative answers plus a Fallback topic can produce **duplicates**. Pick one leader, narrow the other.

COMPOSITION · PUTTING TOPICS TOGETHER

Topics compose — some patterns work, others age badly.

Four patterns the studio supports. Each has a sweet spot — and a careful moment when it stops scaling.

01 · LINEAR

One topic, one path

Trigger → ask → act → close. The simplest shape.

Use when

One intent, one outcome. The first topic anyone writes.

Limit

Three branches in — it's time to split.

02 · ROUTED

A triage topic hands off

One root topic reads intent; branches route to siblings.

Use when

Service-desk fan-out, category-by-category support.

Limit

Too many siblings and the route stops reading.

03 · COMPOSITE

Topics call topics

A topic uses Redirect to send the user down a sibling path.

Use when

A long form, two distinct intents, one shared warm-up.

Limit

Don't loop two topics forever. The user loses the thread.

04 · GENERATIVE

Many topics, one orchestrator

Generative orchestration picks across topics by intent.

Use when

Open-ended Q&A; varied wording; mod surface area.

Limit

Pair with a real fallback — not a polite "I don't know".

LIFECYCLE · FROM DRAFT TO RETIRED

A topic has a life. Don't skip the last two stages.

Topics move through six stages — the last two are the ones most teams forget.

01
Draft
Name, describe, save.

→

02
Test
In the Test pane.

→

03
Stage
Package into a solution.

→

04
Publish
One channel first.

→

05
Tune
Read analytics; re-publish.

→

06
Retire
Disable · archive · keep the audit. Don't leave a zombie.

Retiring is part of the work. Hand over a slack agent and the next owner inherits debris.

TWO HABITS

- **Stage 02 reads the analytics; 02 also reads the conversation map.**
- **Stage 06 closes the loop. Without it, the agent rots.**

THE CYCLE IS NOT OPTIONAL

A draft is not a topic.
An unpublished topic is risk.

USEFUL FOR

Naming each topic in the same dialect — **verb + object** — keeps the list scannable when you have fifteen of them.

OBSERVING TOPICS

A topic is a wire — you can read the current.

Each topic emits signals — per-session traces, escalation rates, unmatched queries, ratings. The wiring below is where to read them.

TOPIC AT RUNTIME

Each topic

A node on the canvas becomes a name on the trace, a row in the sessions table, and a category in the analytics page.

READ 01

Sessions table

Per-session trace · matched topic · time · satisfaction.

READ 02

Topic analytics

Sessions, escalation, abandonment.

READ 03

Unmatched queries

Utterances that hit no topic; the most valuable list.

READ 04

Prompt surface

What the orchestrator actually saw.

THREE SIGNALS A TOPIC AUTHOR SHOULD CHASE

- **Unmatched rate**

  Per topic. Aim down, week over week.

- **Escalation rate**

  Are topics handing off too often, or too rarely?

- **CSAT per topic**

  If it is a 4.6, the topic reads well in the lab but lands cold in life.

LAB · BUILD WITH TOPICS

Author topics end-to-end — trigger, generative, Power Fx.

Eight steps. Three checkpoints. One conversation map at the end. Use whatever channel is convenient for testing — the demo website is fine.

01 AUTHOR
Create the topic
Name it — verb + object. Add 5–10 phrases.

02 PHRASE
Author 5–10 trigger phrases
Real wording; varied verbs; specific.

03 ENABLE
Switch on generative
Generative orchestration on. Test before, after.

04 BRANCH
Add a condition
One variable, one yes/no, two branches.

05 WIRE
Variables + slot filling
Author a Topic variable; map an entity.

06 COMPUTE
A Power Fx expression
Set a global; format a string; decide.

07 FALLBACK
Author the catch-all
Log + redirect or escalate.

08 RUN
Test, publish, observe
Read analytics. Tune. Re-publish.

THREE CHECKPOINTS

AFTER 04
Path covered?
Every branch tested with a real utterance. None skipped.

AFTER 06
Variables traceable?
Open sessions table; every value visible.

AFTER 07
Fallback promising?
Type a mismatch. Read what came back.

A USEFUL LOOP

Test, read, tune — repeat.
Publish once.